

Reviewer Pack — Fourier Coefficient Sum Separation for Read-Twice Branching ...

Entry e0efe9716b37 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 15:54:51 UTC

Fourier Coefficient Sum Separation for Read-Twice Branching Programs

Entry ID: e0efe9716b37 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 15:54:51 UTC

1. Conjecture

Field A (mathematical branch): FOURIER_ANALYSIS **Field B** (complexity object): BRANCHING_PROGRAMS

Statement:

For any read-twice branching program P computing the inner product mod 2 function IP_2 on n variables, the sum of the absolute values of the Fourier coefficients of the function computed by P is $\Omega(\sqrt{n})$, whereas for any function that can be computed by a read-once branching program, this sum is $O(\log n)$.

Rationale (proposer’s reasoning):

The Fourier coefficients of IP_2 exhibit a structure that requires a larger read-twice branching program to capture, as reflected by the sum of their absolute values, which is significantly larger than for functions with simpler Fourier structures computable by read-once programs.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 11e9e6284ee1d201

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_read_twice_branching_program(n):
    program = []
    for i in range(2**n):
        if i % 2 == 0:
            program.append((i, (1, 0)))
        else:
            program.append((i, (0, 1)))
    return program

def generate_read_once_branching_program(n):
    program = []
    for i in range(2**n):
        if i % 2 == 0:
            program.append((i, (1,)))
        else:
            program.append((i, (0,)))
    return program

def fourier_coefficient_sum(program, n):
    sum_coeffs = 0
    for state, (a, b) in program:
        sum_coeffs += abs(a) + abs(b)
    return sum_coeffs

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])

    ip2_twice = generate_read_twice_branching_program(n)
    ip2_once = generate_read_once_branching_program(n)

    ip2_twice_sum = fourier_coefficient_sum(ip2_twice, n)
    ip2_once_sum = fourier_coefficient_sum(ip2_once, n)
```

```

metric_name = "Fourier Coefficient Sum"
metric_value_twice = ip2_twice_sum
metric_value_once = ip2_once_sum

conjecture_holds_twice = metric_value_twice >= math.sqrt(n)
conjecture_holds_once = metric_value_once <= math.log(n, 2)

counterexample_twice = "" if conjecture_holds_twice else "read-twice program"
counterexample_once = "" if conjecture_holds_once else "read-once program"

return {
    "metric_name": metric_name,
    "metric_value": {"twice": metric_value_twice, "once": metric_value_once},
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds_twice and conjecture_holds_once,
    "counterexample": f"read-twice: {counterexample_twice}, read-once: {counterexample_once}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_twice_sum = sum(r["metric_value"]["twice"] for r in results)
    total_once_sum = sum(r["metric_value"]["once"] for r in results)
    mean_twice = total_twice_sum / len(results)
    mean_once = total_once_sum / len(results)
    std_twice = math.sqrt(sum((r["metric_value"]["twice"] - mean_twice) ** 2 for r in results) / len(results))
    std_once = math.sqrt(sum((r["metric_value"]["once"] - mean_once) ** 2 for r in results) / len(results))

    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.8:
        result = f"RESULT: SUPPORTED mean_twice={mean_twice} std_twice={std_twice} support_fraction={support_fraction}"
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        result = f"RESULT: FALSIFIED counterexample=\"read-twice or read-once program\" first_failing_seed={first_failing_seed}"
    else:
        result = "RESULT: INCONCLUSIVE support_fraction too low"

    print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cf3c361.py", line 76, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cf3c361.py", line 50, in run_trial
    ip2_once_sum = fourier_coefficient_sum(ip2_once, n)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cf3c361.py", line 38, in fourier_coeff
    for state, (a, b) in program:
                ^^^^^^

```

ValueError: not enough values to unpack (expected 2, got 1)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unpacking error, preventing data collection. Support condition cannot be evaluated without successful trials. | next: Fix the unpacking error in the fourier_coefficient_sum function by verifying the program’s state structure

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	113804
2	propose	ollama_remote	qwen3:8b	0	0	148835
3	preregistration	ollama_remote	qwen3:8b	0	0	24073
4	novelty	ollama_remote	qwen3:8b	0	0	20770
5	novelty	ollama_remote	qwen3:8b	0	0	16479
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13252
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9942
8	judge	ollama_remote	qwen3:8b	0	0	17833

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 364988 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/e0efe9716b37.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e0efe9716b37.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e0efe9716b37.tar.gz` (if generated)